

SWR302-LLM for Unit Test Case Generation

*Group 5 - 16/07/2026

Nguyen Hoang Hung	Le Quang Thang	Nguyen Huu Khanh	Dang Nguyen Quoc	Trinh Duy Khang
<i>PL - SE180765</i>	<i>MS - SE196526</i>	Duy	Huy	<i>RW - SE183700</i>
		<i>DG - SE196633</i>	<i>LR - SE196510</i>	

Abstract—This study empirically compares GPT-4o-mini (zero-shot prompting) and Pynguin (search-based) for automated unit test generation on 50 Python functions. While GPT-4o-mini achieved a near-perfect completion success rate, it yielded a median Branch Coverage of 0% due to severe hallucinations—predominantly testing fabricated mock objects instead of target code. Conversely, Pynguin achieved a median Branch Coverage of 50%. Interestingly, both approaches recorded a Mutation Score of 0%. Pynguin’s anomaly stems from wrapping failing tests with xfail decorators, causing injected mutants to survive. Findings indicate search-based techniques dominate in structural coverage, highlighting the necessity for hybrid test generation frameworks.

Index Terms—Large Language Models, Automated Unit Testing, Python, Pynguin

I. INTRODUCTION

Writing and creating test units manually is time-consuming and prone to errors, requiring developers to design test cases that include various execution scenarios, boundary conditions, and exceptions. Under time constraints and limited resources, this task is often overlooked or only superficially addressed, resulting in many software projects with weak test suites, making maintenance more difficult and reducing software quality.

Large language models (LLMs) have become a popular research topic in software engineering, primarily because they are quite good at understanding and generating source code. They are now applied to a wide range of software development tasks, such as code completion, bug detection, code summarization, and automated testing. Among these, creating automated test units has become quite common, as writing tests manually requires significant developer effort and is not easy to do well. In real-world projects, developers need to design test cases that include various execution scenarios, boundary cases, and exceptions—something difficult to achieve manually under time constraints, which reduces software quality and increases maintenance costs later on. Previous research has shown that LLM can quite effectively support the creation of automated test units, generating reasonable test cases with minimal human intervention, demonstrating code comprehension and reasoning capabilities. Various techniques have also been tested to improve the quality of generated tests, such as hint strategies, feedback-based approaches, and multi-agent collaboration. Papers by authors such as Wang, Tang,

Ouédraogo, Ryan, Guerino, Vincenzi, Aminata, and Huang all report promising results using metrics like Branch Coverage, Breakpoints, and Completion Success Rate, indicating that LLM has real potential for this task.

Despite its widespread application in this field, several gaps remain. Different studies on datasets, metrics, recommendation strategies, and test setups make direct comparisons difficult. Most existing work also focuses on advanced recommendation techniques or custom LLM frameworks, while lightweight LLMs in simple zero-shot setups receive less attention. More importantly, there is currently very little empirical evidence comparing GPT-4o mini with Pynguin—a widely used tool for automated unit testing for Python—in a consistent evaluation setup. This makes it worthwhile to investigate how well GPT-4o mini actually performs in generating Python unit tests.

Based on these gaps, this study evaluates GPT-4o mini for generating zero-shot unit tests on Python functions with varying levels of cycle complexity. The generated tests were evaluated using Branch Coverage, Breakpoints, Precision, and Test Generation Time. GPT-4o mini was also compared with Pynguin to see which method performed better in this setup. Specifically, this study contributes the following points:

- Experimental evaluation of GPT-4o mini for generating untrained unit tests on Python functions with different cycle complexities.

- Comparison between GPT-4o mini and Pynguin using Branch Coverage, Breakpoints, Precision, and Test Generation Time.

- Discussion of the strengths and weaknesses of automated unit test generation methods based on LLM versus traditional methods for Python.

The remainder of this paper is organized as follows. Section II reviews related work on LLM-based and search-based unit test generation. Section III presents the research methodology, including the benchmark dataset, experimental procedure, evaluation metrics, and statistical analysis. Section IV reports the experimental results. Section V discusses the findings and provides an error analysis. Section VI outlines the threats to validity. Finally, Section VII concludes the paper and suggests directions for future work.

II. RELATED WORK

A. LLM-based Unit Test Generation

Several recent studies have evaluated the ability of large language models (LLMs) to automatically generate unit tests

on real-world code. Huang et al. introduced the ULT benchmark, consisting of 3,909 Python functions ($CC \geq 10$) built to prevent data contamination, and showed that the Pass@5 accuracy on the clean benchmark was only 12.57

The common point of this research group is: (1) LLM performance degrades sharply as method complexity increases (HITS); (2) Less than half of the studies report mutation scores alongside coverage, although high coverage does not necessarily mean good error detection; (3) The majority focus on Java, while Python—the language used in this study—gives low results and is less systematically controlled for function complexity; and (4) The risk of data contamination can significantly skew the numbers published on popular benchmarks.

B. Search-Based Software Testing (Baseline)

Alongside the LLM approach, many studies use search-based testing tools (SBST) such as EvoSuite or Pynguin as a baseline for comparison. Tang et al. directly compared ChatGPT with EvoSuite on 207 Java classes, showing that EvoSuite significantly outperformed ChatGPT in statement coverage (74.2

These results are inconsistent – SBST may win or lose to LLM depending on the dataset and metric – but they all suggest an important point: high coverage does not guarantee a correspondingly high mutation score, and vice versa. Notably, none of the studies in this group constructed a set of cyclomatically complex (CC)-controlled stratified functions with fixed quotas; both SymPrompt and Guerino & Vincenzi derived functions from open-source code with uncontrolled CC distributions, potentially leading to confounded model complexity and performance.

C. Research Positioning

Unlike the studies above, this is the first study to specifically evaluate **GPT-4o-mini** (instead of full GPT-4/GPT-4o) against the baseline **Pynguin** (DYNAMOSA algorithm) on a set of 50 Python functions stratified in a controlled manner across three cyclomatic complexity levels (CC 5–8, CC 9–12, CC 13–15) with fixed quotas of 40

III. METHODOLOGY

This study conducts an empirical comparison between GPT-4o mini and Pynguin for automated Python unit test generation. Both approaches were evaluated under the same experimental environment to ensure a fair comparison. GPT-4o mini generated unit tests using a zero-shot prompting strategy without additional examples or prompt refinement, while Pynguin generated test suites using its default search-based algorithm. The methodology is designed to ensure fairness, reproducibility, and rigorous quantitative evaluation.

A. Research Design

The experiment follows a controlled comparative design in which both GPT-4o mini and Pynguin generate unit tests for the same benchmark under identical runtime conditions. All generated test suites are evaluated using identical measurement tools and execution environments.

To reduce threats caused by model drift, all GPT-generated tests were produced using the fixed model version `gpt-4o-mini-2024-07-18`. All experiments were executed using the same Python interpreter, package versions, and operating environment.

B. Benchmark Dataset

The benchmark consists of 50 independent Python functions collected from open-source repositories. The selected functions represent different control-flow structures and cyclomatic complexity levels to provide diverse testing scenarios.

Functions requiring external dependencies, graphical user interfaces, databases, or network communication were excluded to minimize environmental bias and improve reproducibility.

A function was included if it satisfied the following conditions:

- 1) It was implemented in Python.
- 2) It could be executed as a standalone function without requiring unavailable external services.
- 3) It contained meaningful control-flow logic suitable for unit testing.
- 4) Its cyclomatic complexity fell within the target range defined for this study.

Cyclomatic complexity was measured before the experiment to ensure that the benchmark covered functions with varying structural complexity. After applying the inclusion and exclusion criteria, the final benchmark contained 50 Python functions representing diverse control-flow patterns and application scenarios.

C. Experimental Procedure

The experiment was performed according to the following workflow:

- 1) Select the benchmark functions.
- 2) Generate unit tests using GPT-4o mini.
- 3) Generate unit tests using Pynguin.
- 4) Execute all generated test suites under identical runtime conditions.
- 5) Measure Compilation Success Rate, Branch Coverage, Mutation Score, and Test Generation Time.
- 6) Perform statistical analysis and compare both approaches.

a) *Prompt*: GPT-4o mini received exactly the same prompt for every benchmark function.

Generate pytest unit tests for the following Python function:
Requirements:

- Use pytest.
- Include normal, boundary and edge cases.
- Output only Python code.

Function:
<source_code>

b) *Error Handling*: Several mechanisms were implemented to handle API failures during test generation.

- Empty responses were marked as invalid.
- Rate-limit errors were handled using exponential backoff.
- Unexpected API failures were logged for manual inspection.

D. Evaluation Metrics

Four evaluation metrics were used to assess the generated test suites.

- **Compilation Success Rate (CSR)**

Measures the proportion of generated test suites that successfully compile without syntax errors.

$$CSR = \frac{N_{success}}{N_{total}} \times 100$$

where $N_{success}$ denotes the number of compilable test suites and N_{total} denotes the total number of generated test suites.

- **Branch Coverage (BC)**

Measures the percentage of executed control-flow branches in the source code. Coverage was collected using `coverage.py` (v7.x) with the `-branch` option under `pytest`. Branch Coverage serves as the primary effectiveness metric.

- **Mutation Score (MS)**

Evaluates the fault-detection capability of generated test suites. Mutation testing was performed using the `cosmic-ray` (v8.x) framework. Mutation Score is computed as

$$MS = \frac{M_{killed}}{M_{total}} \times 100$$

where M_{killed} denotes the number of killed mutants and M_{total} denotes the total number of generated mutants [?].

- **Test Generation Time**

Measures the elapsed time required to generate a complete unit test suite for each benchmark function.

E. Metric Computation Procedure

Compilation Success Rate was computed by attempting to compile each generated test file using Python’s built-in `py_compile` module. A generated test suite was considered successful if it compiled without raising any syntax exception.

Branch Coverage was measured only for compilable test suites. Each generated test suite was executed together with its corresponding source function inside an isolated working directory using `pytest` and `coverage.py` configured with the `-branch` option. The percentage of covered branches was extracted directly from the generated coverage report.

During this process, we observed that some GPT-4o mini generated tests did not invoke the original source function. Instead, the model occasionally generated self-contained implementations or mock functions and tested those implementations rather than the supplied source code. Consequently, the original source module was never executed, resulting in

0% Branch Coverage. These cases were explicitly recorded (flagged as *source not imported*) and treated as a meaningful experimental finding rather than a measurement artifact.

Mutation Score was measured using the `cosmic-ray` mutation testing framework. For each benchmark function, artificial mutants were generated automatically. A mutant was considered killed if at least one generated test failed after mutation; otherwise, it survived. Mutation Score was computed as the percentage of killed mutants.

Test Generation Time was measured from the moment a generation request was submitted until the complete test suite was returned. For Pynguin, timing covered the entire automatic search process until test generation finished.

F. Statistical Analysis Plan

To ensure the reliability of the experimental findings, all statistical analyses were performed using Python’s `scipy.stats` and `cliffs-delta` libraries.

a) *Statistical Tests*: Branch Coverage and Mutation Score are bounded metrics that generally follow non-normal distributions. Therefore, differences between GPT-4o mini and Pynguin were analyzed using the non-parametric Wilcoxon Signed-Rank Test [?].

Compilation Success Rate was additionally evaluated using an exact binomial test against the predefined acceptance threshold of 80%.

b) *Significance Level*: Because multiple hypothesis tests were conducted on the same benchmark dataset, the Bonferroni correction was applied to control the family-wise error rate.

$$\alpha_{adj} = \frac{0.05}{3} = 0.0167$$

A result is considered statistically significant only when

$$p < 0.0167.$$

c) *Effect Size*: Besides statistical significance, practical significance was quantified using Cliff’s Delta (δ), which is appropriate for ordinal and non-parametric paired data.

Following Romano et al. [?], the interpretation thresholds are:

- $|\delta| < 0.147$: Negligible
- $0.147 \leq |\delta| < 0.330$: Small
- $0.330 \leq |\delta| < 0.474$: Medium
- $|\delta| \geq 0.474$: Large

G. Reliability of Ground Truth

To improve the reliability of the manual assessment process, a pilot annotation was independently performed by two annotators. Inter-rater agreement was evaluated using Cohen’s Kappa. The obtained agreement reached $\kappa = 0.80$, exceeding the predefined acceptance threshold of 0.70, indicating substantial agreement and supporting the reliability of the evaluation process.

To ensure reproducibility, the complete benchmark dataset (50 Python functions), generated test suites, measurement scripts, and analysis code have been publicly released at

IV. RESULTS

The evaluation was conducted on $N=50$ valid functions from the dataset. During the Compilation Success Rate (CSR) phase, 2.0% of the GPT-generated test suites (1 out of 50) returned syntax/compilation errors; however, all $N=50$ functions were included in the overall statistical analysis to accurately reflect the tools' reliability. To account for multiple comparisons, the Bonferroni-corrected significance level is set at $\alpha_{adj} = 0.0167$.

A. RQ1: Can LLM (GPT) generate syntactically correct test cases that achieve higher Branch Coverage than the baseline (Pynguin)?

Table I shows the Compilation Success Rate (CSR) and Branch Coverage (BC) scores for each strategy.

For CSR, the GPT approach achieved a success rate of 98.0% (49/50). The exact binomial test yields $p = 1.9268 \times 10^{-4}$. Therefore, we reject H_0 for CSR, confirming that GPT significantly exceeds the 80% baseline threshold.

For BC, the median score is 0.0% for GPT and 50.0% for Pynguin. The Wilcoxon signed-rank test yields $p = 1.0000$, Cliff's Delta $\delta = -0.70$ (large). Therefore, we fail to reject H_0 for Branch Coverage. Figure ?? illustrates the distribution of Branch Coverage scores across conditions.

TABLE I: Compilation Success Rate and Branch Coverage (RQ1)

Method	CSR	BC	p	Effect
Pynguin	—	50.0%	—	—
CSR:				
			1.93×10^{-4}	
BC:				
				$\delta = -0.70$
GPT-4o mini	98.0% (49/50)	0.0%	1.000	Large

Note: Branch Coverage is reported using the median because the data are non-normally distributed.

B. RQ2: Do test cases generated by LLM (GPT) achieve a higher Mutation Score compared to the baseline (Pynguin)?

Table II shows the Mutation Score (MS) for each condition.

The median MS is 0.0% for both the GPT approach and the Pynguin baseline. Due to both datasets consisting entirely of zeros (zero variance), the Wilcoxon signed-rank test could not be computed mathematically. The effect size evaluation yields Cliff's Delta $\delta = 0.00$ (negligible).

Therefore, we fail to reject H_0 . Neither tool successfully passed the baseline validation required to detect injected mutants. Figure ?? illustrates the distribution of Mutation Scores across conditions.

TABLE II: Mutation Score (RQ2)

Condition	MS (Median)	p-value	Effect size
Pynguin (Baseline)	0.0%	—	—
Approach GPT	0.0%	N/A	$\delta = 0.00$ (negligible)

V. DISCUSSION

A. Explanation of Main Findings and Error Analysis

Our evaluation reveals a stark contrast between the syntactic correctness of LLM-generated code and its actual semantic utility in a testing environment.

For RQ1, the GPT approach achieved a near-perfect Compilation Success Rate (98.0%), successfully bypassing syntax and structural errors. However, this high compilation rate did not translate into structural coverage. Our results suggest that the 0.0% median Branch Coverage for GPT may be attributed to profound semantic hallucinations during the zero-shot generation process. An in-depth error analysis of the worst-performing cases reveals a consistent pattern: instead of importing and invoking the target functions from the provided functions/ directory, the LLM frequently hallucinated and defined its own mock classes within the test file itself. Consequently, the tests executed successfully and passed, but they only exercised the dummy mock logic rather than the actual target source code.

For RQ2, both GPT and Pynguin achieved a median Mutation Score of 0.0%. The failure of GPT is directly linked to the aforementioned hallucinations, leading to baseline validation failures (e.g., `AssertionError`, `NameError`) when integrated with the Cosmic-Ray mutation framework. Pynguin, despite achieving a 50.0% median Branch Coverage, also failed to kill any injected mutants. Error analysis indicates that Pynguin frequently struggled with complex logic instantiation. Rather than generating robust assertions to verify outputs, Pynguin's genetic algorithm often generated test cases that would inherently crash, subsequently applying the `@pytest.mark.xfail` (expected failure) decorator to bypass test suite termination. As a result, when mutants altered the core logic, the tests still crashed and were marked as "passed" by the `xfail` handler, rendering the test suite completely blind to semantic faults.

VI. THREATS TO VALIDITY

A. Internal Validity

The experiment relied on GPT-4o mini and Pynguin under fixed execution settings. Although both approaches were evaluated on the same benchmark and environment, implementation details such as prompt wording, model updates, and tool configurations may influence the generated test suites. To mitigate this threat, all experiments were conducted using identical benchmark functions, a consistent execution environment, and the same evaluation pipeline.

B. Construct Validity

Branch Coverage and Mutation Score were selected as the primary evaluation metrics because they are widely adopted in automated software testing research. However, these metrics do not fully capture all aspects of test quality, such as readability, maintainability, or the ability to detect domain-specific defects. Furthermore, the Mutation Score in this study was affected by the current `xfail` wrapping issue in the mutation-testing

pipeline, which resulted in a score of 0% for both approaches. Consequently, Branch Coverage served as the primary indicator for comparing testing effectiveness.

C. External Validity

The benchmark consisted of 50 Python functions collected from publicly available GitHub repositories. Although the selected functions covered multiple control-flow patterns and complexity levels, they may not fully represent large-scale industrial software systems or projects with extensive external dependencies. Therefore, the findings should be generalized cautiously beyond the evaluated benchmark.

D. Conclusion Validity

The statistical conclusions depend on the selected benchmark size and the applied non-parametric statistical tests. To improve reliability, the study employed paired statistical analysis using the Wilcoxon signed-rank test together with Cliff's Delta to assess both statistical significance and effect size. Nevertheless, a larger benchmark and additional replication studies would further strengthen the confidence in the reported findings.

VII. CONCLUSION

This study compares GPT-4o mini and Pynguin in generating automated test units for Python functions with varying levels of cycle complexity. The results show that Pynguin achieved higher branch coverage than GPT-4o mini under the same setup. Furthermore, both methods had a 0% mutation rate, indicating that higher coverage doesn't necessarily translate into better error detection. This highlights a common limitation between LLM-based and traditional test generation methods: improving test quality and considering execution context still require further attention in future studies.

REFERENCES

- [1] Y. Yuan, X. Wang, and Y. Lou, "An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2024.
- [2] M. Wang, Y. Tang, and X. Li, "A System for Automated Unit Test Generation Using Large Language Models and Assessment of Generated Test Suites," *Journal of Systems and Software*, vol. 215, 2024.
- [3] M. Wang, Y. Tang, and H. Liu, "Method Slicing for Improving LLM-Based Automated Unit Test Generation," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [4] J. Yang, L. Chen, and K. Huang, "An Evaluation Framework for Large Language Model-Based Unit Test Generation," *Empirical Software Engineering*, vol. 29, no. 6, 2024.
- [5] A. Xu, H. Zhang, and J. Chen, "AI-Powered Multi-Agent Framework for Automated Unit Test Case Generation," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2025.
- [6] P. Baskaran, S. Kumar, and R. Singh, "Prompt-Driven Efficient Unit Test Generation Using Large Language Models," *IEEE Access*, vol. 13, 2025.
- [7] F. Chang, R. Zhao, and X. Wu, "What Inputs Drive Effective Large Language Model-Based Unit Test Generation?," 2026.
- [8] M. Lukaszczuk, G. Fraser, and A. Panichella, "Pynguin: Automated Unit Test Generation for Python," in *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*, 2020.

- [9] M. Lukaszczuk, G. Fraser, and A. Panichella, "Pynguin: Automated Unit Test Generation for Python," in *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*, 2020.
- [10] OpenAI, "GPT-4o mini," OpenAI, Jul. 2024. [Online]. Available: <https://openai.com/index/gpt-4o-mini/>
- [11] T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [12] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, Sept.–Oct. 2011.